RAG University Assistant - Implementation Plan

Phase 1: Environment Setup (Day 1)

Prerequisites

- Python 3.9+
- 8GB+ RAM recommended
- Basic understanding of Python and virtual environments

Setup Steps

bash

```
# Create project structure
mkdir uni-assistant
cd uni-assistant
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install core dependencies
pip install scrapy beautifulsoup4 requests
pip install langchain langchain-community
pip install sentence-transformers
pip install chromadb
pip install ollama
pip install streamlit
pip install apscheduler
```

Project Structure

```
uni-assistant/
├── data/
│   ├── raw/              # Scraped HTML/PDFs
│   ├── processed/        # Cleaned text chunks
│   └── vectorstore/      # ChromaDB storage
├── src/
│   ├── scraper.py        # Web scraping logic
│   ├── processor.py      # Text processing & chunking
│   ├── embedder.py       # Embedding generation
│   ├── retriever.py      # RAG retrieval logic
```

```
│   ├── llm_handler.py    # LLM interaction
│   └── app.py            # Streamlit frontend
├── config.py             # Configuration settings
└── requirements.txt
```

Phase 2: Data Collection (Days 2-3)

Goal

Scrape your university website and extract relevant information.

Implementation

Step 2.1: Create a Simple Web Scraper

- Start with your school's main pages (admissions, programs, FAQs)
- Extract text content and preserve URLs for citations
- Handle pagination and navigation
- Save raw HTML/text to `data/raw/`

Step 2.2: Document Parsing

- Add support for PDFs (course catalogs, handbooks)
- Extract text from downloaded documents
- Preserve metadata (title, URL, last modified date)

Key Considerations:

- Start small: 50-100 pages initially
- Respect robots.txt
- Add delays between requests (1-2 seconds)
- Handle errors gracefully

Phase 3: Text Processing & Embedding (Days 4-5)

Goal

Convert raw text into searchable vector embeddings.

Implementation

Step 3.1: Text Chunking

- Split documents into 500-1000 token chunks
- Preserve context with overlap (100-200 tokens)
- Keep metadata with each chunk (source URL, section)

Step 3.2: Generate Embeddings

- Use `sentence-transformers/all-MiniLM-L6-v2` (lightweight, 384 dimensions)
- Alternative: `sentence-transformers/all-mpnet-base-v2` (better quality, 768 dimensions)
- Generate embeddings for all chunks
- Store in ChromaDB with metadata

Step 3.3: Setup Vector Database

- Initialize ChromaDB persistent storage
- Create collection with metadata filtering
- Test basic similarity search

Phase 4: LLM Setup (Day 6)

Goal

Get a local LLM running for response generation.

Implementation

Step 4.1: Install Ollama

```bash
# Download from https://ollama.ai
# Pull a model
ollama pull llama3.2:3b  # Faster, less RAM
# or
ollama pull mistral:7b   # Better quality
```

Step 4.2: Test LLM Locally

- Verify model responds to prompts
- Test response time and quality
- Experiment with system prompts

Phase 5: RAG Pipeline (Days 7-8)

Goal

Connect retrieval and generation into a working pipeline.

Implementation

Step 5.1: Build Retriever

- Accept user query
- Generate query embedding
- Retrieve top-k similar chunks (k=3-5)
- Return chunks with metadata

Step 5.2: Prompt Engineering Create a system prompt template:

```
You are a helpful university assistant for [University Name].
Answer questions based ONLY on the provided context.
If you don't know the answer, say so and suggest contacting admissions.

Context:
{retrieved_chunks}

Question: {user_question}

Answer:
```

Step 5.3: Generate Responses

- Send prompt to Ollama
- Stream responses for better UX
- Include source citations in responses

Phase 6: User Interface (Days 9-10)

Goal

Create a simple chat interface with Streamlit.

Implementation

Step 6.1: Basic Chat UI

- Message input box
- Conversation history display
- Source citations as expandable sections

Step 6.2: Add Features

- Clear conversation button
- Loading indicators
- Error handling
- Copy response button

Step 6.3: Conversation Memory

- Store last 5-10 messages
- Include in context for follow-up questions
- Save to SQLite for persistence (optional)

Phase 7: Testing & Refinement (Days 11-12)

Testing Checklist

- [ ] Test with common student questions
- [ ] Verify citation accuracy
- [ ] Check response quality
- [ ] Test edge cases (unclear questions, out-of-scope queries)
- [ ] Measure response time
- [ ] Test with limited context

Optimization

- Adjust chunk size if needed
- Tune retrieval (number of chunks)
- Refine system prompt
- Experiment with different embeddings models

Phase 8: Automation & Scheduling (Day 13)

Goal

Automate periodic website scraping and index updates.

Implementation

- Use APScheduler for periodic tasks
- Schedule weekly scraping
- Implement incremental updates
- Add logging for monitoring

Minimal Viable Prototype (MVP) Checklist

Week 1-2 Deliverable:

- ✅ Scrapes 50-100 university pages
- ✅ Stores data in ChromaDB
- ✅ Retrieves relevant context for queries
- ✅ Generates responses using local LLM
- ✅ Simple Streamlit chat interface
- ✅ Shows source citations

Optional Enhancements (Later):

- Advanced filtering (by department, topic)
- Multi-language support
- Analytics dashboard

- API endpoint for integration
- Mobile-responsive UI

Estimated Resource Requirements

Storage: 500MB - 2GB (depends on website size) **RAM:** 8GB minimum (4GB for LLM, 2GB for embeddings) **Processing:** First indexing may take 30-60 minutes **Query Response:** 2-5 seconds per question

---

Cost Breakdown (All Free!)

Component	Open-Source Tool	Cost
Web Scraping	Scrapy/BeautifulSoup	$0
Embeddings	sentence-transformers	$0
Vector DB	ChromaDB	$0
LLM	Ollama (Llama/Mistral)	$0
Frontend	Streamlit	$0
Hosting	Local/Render Free Tier	$0

Next Steps After MVP

1. Deploy to Render/Railway (free tiers)
2. Add user feedback mechanism
3. Create documentation
4. Record demo video for portfolio
5. Write blog post about the project
6. Share on GitHub with detailed README

Portfolio Impact Tips

Highlight in Your Portfolio:

- Problem solved (student information accessibility)
- Technical architecture diagram
- Challenges overcome (handling large documents, LLM hallucinations)
- Metrics (response time, accuracy, pages indexed)
- Demo video showing real queries
- GitHub with clean code and documentation